fn match_truncations

Michael Shoemate (@Shoeboxam)

November 8, 2026

This proof resides in **"contrib"** because it has not completed the vetting process.

Proves soundness of `match_truncations` in `mod.rs` at commit 7b5224f (outdated[1]).

1 Hoare Triple

Precondition

Compiler-verified

Types matching pseudocode.

Caller-verified

None

Pseudocode

```
def match_truncations(
    plan: DslPlan, identifier: Expr
) -> tuple[DslPlan, Vec[Truncation], Vec[Bound]]:
    truncations = []
    bounds = []

    allowed_keys = match_group_by_truncation(
        plan, identifier
    )  #
    if allowed_keys:
        input, truncate, new_bound = allowed_keys
        plan = input  #
        truncations.append(truncate)
        bounds.append(new_bound)  #
        allowed_keys = new_bound.by  #

    # match until not a filter truncation
    while isinstance(plan, Truncation.Filter):  #
        input, predicate = plan.input, plan.predicate
        new_bounds = match_truncation_predicate(
            predicate, identifier
        )  #
        if new_bounds is None:
            break
```

[1] See new changes with `git diff 7b5224f..e28f9c7 rust/src/transformations/make_stable_lazyframe/truncate/matching/mod.rs`

```
25
26          # When filter truncation is behind a groupby truncation,
27          # if the groupby group keys don't cover the filter truncation keys,
28          # then the groupby aggs can overwrite the filter truncation keys,
29          # invalidating the filter truncation bounds.
30          if allowed_keys is not None:  #
31              for bound in new_bounds:
32                  if not bound.by.is_subset(allowed_keys):
33                      raise f"Filter truncation keys ({bound.by}) must be a subset of groupby
    truncation keys ({allowed_keys})."
34
35          plan = input  #
36          truncations.append(Truncation.Filter(predicate))
37          bounds.extend(new_bounds)  #
38
39      # just for better error messages, no privacy implications
40      if (
41          match_group_by_truncation(plan, identifier) is not None
42      ):  #
43          raise Exception("groupby truncation must be the last truncation in the plan.")
44
45      # since the parse descends to the source,
46      # truncations and bounds are in reverse order
47      truncations.reverse()  #
48      bounds.reverse()  #
49
50      return plan, truncations, bounds
```

Postcondition

Theorem 1.1 (Postcondition)**.** For any choice of LazyFrame plan, returns the plan with the truncations removed, the truncations that were removed in the order they are applied, and per-id bounds on row and/or group contributions using the original grouping expressions. If a groupby truncation is present, its caller must express these bounds in terms of the materialized output columns before using them as output distance bounds.

Proof. The algorithm maintains three invariants:

- `input` is the LazyFrame plan with truncations removed
- `truncations` is a list of truncations in reverse-order
- `bounds` is a list of per-id bounds on row and/or group contributions

In order to ensure that all per-id bounds remain valid after successive truncations, the group-by truncation may only be applied last in the truncation pipeline, as the group-by truncation rewrites all columns in the data, and potentially overwrites user identifiers.

Since parsing the query plan happens in reverse order, the algorithm starts by attempting to parse a group-by truncation on line 9. By the postcondition of `match_group_by_truncation`, if the group-by truncation is present, it will be returned as a tuple of `input`, (the execution plan with the group-by truncation removed), `truncation`, (the group-by truncation) and `per_id_bounds` (the per-id bounds on row contributions). The state of the algorithm is then updated on lines 12-14, maintaining the invariants on `input`, `truncations` and `bounds`.

Another limitation of the group-by truncation is that bounds on row contributions when grouped by a given set of columns are no longer valid if those columns are changed in the group-by truncation. Therefore, `allowed_keys` on line 15 contains the original grouping expressions whose values are materialized by the group-by truncation. This guarantees preservation of their grouping labels, not equivalence when those expressions are re-evaluated on the output. The caller must translate each retained expression to its materialized output column. The subset check is necessary for this correspondence.

If a group-by truncation is not present, no update is made to the state.

The algorithm then attempts to repeatedly parse filter truncations on line 18. By the postcondition of `match_truncation_predicate` on line 22, the return is a list of per-id contribution bounds if the predicate consists solely of truncations, otherwise none. This ensures that the algorithm rejects predicates that contain conditions that are not truncations.

Line 30 checks that the truncation predicate is valid, by ensuring that the truncation predicate is a subset of the allowed keys. The algorithm then updates the state on lines 35-37, maintaining the invariants on `input`, `truncations` and `bounds`. Finally, since the descent through the query plan is in reverse order, line 47 ensures that the truncation order is correct. The check on line 42 rejects a groupby truncation below the already matched stages. This keeps the accepted pipeline limited to filters followed by at most one final groupby, so that no later matched truncation reinterprets the materialized output columns. The reversal on line 48 puts the bounds in the order used by the caller.

Since the invariants on `input`, `truncations` and `bounds` are maintained, and the algorithm only matches through truncations, the postcondition is satisfied. □